

# Sprawozdanie z Projektu: Wizualizacja Drukarki 3D

Szymon Wójcik (198212), Patryk Jakiel (197733)

Kierunek: ACiR 1A

## 1 Wprowadzenie i Opis Projektu

Projekt realizuje interaktywną wizualizację procesu drukowania trójwymiarowych obiektów w czasie rzeczywistym. Aplikacja, stworzona w języku Python z wykorzystaniem bibliotek Pygame i PyOpenGL, umożliwia obserwację wirtualnej głowicy drukującej, która nanosi kolejne warstwy materiału, tworząc predefiniowane modele 3D, takie jak sześciany i kule. Użytkownik może manualnie sterować kluczowymi elementami drukarki (głowicą, suwakiem, blatem) oraz inicjować procesy, takie jak drukowanie, usuwanie wydruku czy powrót do pozycji neutralnej. Wprowadzono także obsługę shaderów w OpenGL, co znacząco poprawia jakość wizualną sceny poprzez dodanie realistycznego oświetlenia i cieni.

## 2 Realizacja Wizualizacji i Wybór Technologii

Początkowo rozważano implementację z użyciem środowiska VPython. Ostatecznie jednak zdecydowano się na kombinację bibliotek Pygame i PyOpenGL. Decyzja ta podyktowana była chęcią głębszego zrozumienia i wykorzystania bardziej niskopoziomowych narzędzi graficznych, co lepiej wpisuje się w cele projektu. Pygame odpowiada za obsługę okna aplikacji, pętlę zdarzeń (wejścia z klawiatury i myszy) oraz renderowanie elementów 2D, takich jak licznik FPS. PyOpenGL jest natomiast fundamentem dla całej grafiki 3D, umożliwiając rysowanie zarówno statycznych elementów drukarki, jak i dynamicznie tworzonych obiektów oraz integrację shaderów.

### 2.1 Przykład porównawczy: tworzenie kostki w VPython i PyOpenGL

Dla zobrazowania różnicy w złożoności między VPython a PyOpenGL przedstawiono przykład stworzenia zwykłej sześciennej kostki o boku 1, umieszczonej w punkcie (0,0,0). Kod w VPython jest niezwykle zwięzły, podczas gdy implementacja w PyOpenGL wymaga ręcznego zdefiniowania wierzchołków i ścian, co daje jednak pełną kontrolę nad procesem renderowania.

#### Kod w VPython

Tworzenie sześcianu sprowadza się do wywołania jednej funkcji `box()`:

```
1 # Import biblioteki
2 from vpython import box
3
4 # Stworzenie kostki jednym poleceniem
5 box(pos=vector(0,0,0), size=vector(1,1,1), color=color.red)
```

#### Kod w PyOpenGL

Ta sama operacja w PyOpenGL wymaga ręcznego zdefiniowania geometrii obiektu za pomocą wierzchołków:

```

1 from OpenGL.GL import *
2
3 vertices = [(-0.5, -0.5, -0.5), (0.5, -0.5, -0.5),
4             (0.5, 0.5, -0.5), (-0.5, 0.5, -0.5),
5             (-0.5, -0.5, 0.5), (0.5, -0.5, 0.5),
6             (0.5, 0.5, 0.5), (-0.5, 0.5, 0.5)]
7
8 faces = [(0,1,2,3), (4,5,6,7), (0,1,5,4),
9          (2,3,7,6), (1,2,6,5), (0,3,7,4)]
10
11 glBegin(GL_QUADS)
12 for face in faces:
13     for vertex in face:
14         glVertex3fv(vertices[vertex])
15 glEnd()

```

## 2.2 OpenGL – Model Graficzny i Układ Współrzędnych

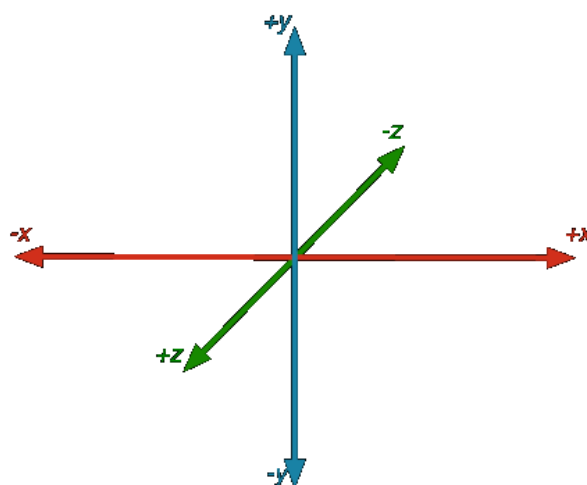
OpenGL (Open Graphics Library) to niskopoziomowe API graficzne wykorzystywane do tworzenia grafiki 2D i 3D. W projekcie pełni ono rolę silnika renderującego, odpowiedzialnego za rysowanie sceny 3D, oświetlenie, kolory, cieniowanie oraz transformacje przestrzenne.

W klasycznym układzie współrzędnych OpenGL przestrzeń 3D opisana jest trzema osiami:

- X – pozioma, skierowana w prawo,
- Y – pionowa, skierowana do góry,
- Z – głębokość sceny, skierowana „do wewnątrz” (w głąb monitora).

Punkt  $(0, 0, 0)$  stanowi środek sceny, wokół którego rozmieszczane są obiekty. Transformacje takie jak `glTranslatef`, `glRotatef` czy `glScalef` pozwalają przemieszczać, obracać i skalować obiekty względem tej przestrzeni.

Dzięki temu można realistycznie odwzorować proces drukowania 3D, odwzorowując ruchy drukarki i pozycję głowicy w rzeczywistej przestrzeni.



Rysunek 1: Układ współrzędnych OpenGL – widok izometryczny

## 3 Struktura i Działanie Aplikacji

### 3.1 Inicjalizacja i Główna Pętla

Aplikacja rozpoczyna działanie w funkcji `main()`. Inicjalizacja obejmuje:

- Uruchomienie bibliotek Pygame (`pygame.init()`, `pygame.font.init()`).
- Utworzenie okna w trybie `DOUBLEBUF | OPENG`L dla płynnego renderowania 3D.
- Konfigurację środowiska OpenGL:
  - `glClearColor(0.8, 0.8, 0.8, 1.0)`: Ustawia jasnoniebieskie tło.
  - `gluPerspective(45, (width/height), 0.1, 500)`: Definiuje perspektywę z kątem widzenia 45 stopni, proporcjami ekranu i płaszczyznami przycinania.
  - `glEnable(GL_DEPTH_TEST)`: Włącza test głębi dla poprawnego renderowania obiektów.
  - `glEnable(GL_LIGHTING)` i `glEnable(GL_LIGHT0)`: Włącza oświetlenie i źródło światła.

Główna pętla (`while True`) odpowiada za odświeżanie sceny, obsługę zdarzeń, aktualizację logiki i renderowanie grafiki z ograniczeniem do 60 klatek na sekundę (`clock.tick(60)`).

### 3.2 Wyświetlanie i Deklaracja Kształtów Podstawowych

Podstawowymi elementami sceny są prostopadłościany i walce, renderowane za pomocą funkcji `rysuj()` i `rysuj_walec()`.

#### 3.2.1 Deklaracja i Renderowanie Sześcianu/Prostopadłościanu

Podstawą każdego prostopadłościanu w scenie (od nóżek stołu po pojedyncze "wydrukowane" kostki) jest zbiór danych geometrycznych:

- **vertices**: Krotka zawierająca 8 krotek, z których każda definiuje współrzędne (x, y, z) jednego wierzchołka sześcianu jednostkowego,
- **edges**: Krotka definiująca 12 krawędzi poprzez łączenie par wierzchołków z listy vertices,
- **faces**: Krotka definiująca 6 ścian poprzez łączenie czterech wierzchołków.
- **Wektory normalne (normals)**: to zbiór 6 trójwymiarowych wektorów, które są prostopadłe do każdej z 6 ścian sześcianu. Ich znaczenie jest kluczowe w kontekście oświetlenia w grafice 3D – pozwalają silnikowi renderującemu obliczyć, jak światło odbija się od powierzchni.

Funkcja `rysuj(pos, size, color)` przyjmuje pozycję, rozmiar i kolor. Jej działanie można opisać następująco:

1. `glPushMatrix()`: Zapisuje aktualną macierz transformacji. Jest to kluczowe dla izolowania transformacji tego obiektu od innych.
2. `glTranslatef(x, y, z)`: Przesuwa układ współrzędnych do pozycji obiektu.
3. `glScalef(sx, sy, sz)`: Skaluje sześcian jednostkowy do pożądaných wymiarów, tworząc prostopadłościan.

4. `glMaterialfv(GL_FRONT_AND_BACK, GL_AMBIENT_AND_DIFFUSE, color)`: Ustawia materiał obiektu dla oświetlenia.
5. `glBegin(GL_QUADS)`: Rozpoczyna rysowanie czworokątów. Pętla iteruje po zdefiniowanych ścianach (`faces`), ustawia kolor (`glColor3fv`) i rysuje każdy wierzchołek (`glVertex3fv`), tworząc wypełnione powierzchnie.
6. `glBegin(GL_LINES)...``glEnd()`: Rysuje czarne krawędzie obiektu, iterując po liście `edges`, co poprawia czytelność modelu.
7. `glEnd()`: Zamyka listę wierzchołków.
8. `glPopMatrix()`: Przywraca poprzednią macierz transformacji, kończąc rysowanie obiektu.

### 3.2.2 Renderowanie Walca

Funkcja `rysuj_walec(pos, wysokosc, promien, kolor)` wykorzystuje bibliotekę GLU:

1. `gluNewQuadric()`: Tworzy nowy obiekt "kwadryki", który jest podstawą do generowania walców, sfer itp.
2. `glRotatef(-90, 1, 0, 0)`: Obraca układ współrzędnych, aby walec był orientowany pionowo.
3. `gluDisk()`: Rysuje dyski (podstawy walca).
4. `gluCylinder()`: Rysuje powierzchnię boczną walca.
5. `gluDeleteQuadric()`: Zwalnia zasoby kwadryki.

Metoda ta generuje gładkie powierzchnie z automatycznie obliczonymi wektorami normalnymi (`GLU_SMOOTH`).

### 3.3 Przechowywanie Wydrukowanych Klocków

Wszystkie "wydrukowane" przez drukarkę klocki są przechowywane w zmiennej `cubes`. Jest to zbiór (`set`), co jest świadomą i efektywną decyzją projektową. Każdy klocek jest reprezentowany jako krotka (`x, y, z`) określająca jego względną pozycję. Użycie zbioru gwarantuje, że **każdy klocek w danej lokalizacji jest unikalny** (nie można dodać dwóch klocków w to samo miejsce) oraz zapewnia bardzo szybkie operacje dodawania i sprawdzania przynależności, co jest istotne przy dużych modelach.

Podczas renderowania, pętla `for x_c, y_c, z_c in cubes`: iteruje po wszystkich zapisanych pozycjach klocków. Dla każdego klocka, jego ostateczna pozycja jest obliczana poprzez dodanie globalnego przesunięcia blatu (`z_pos_blatu`) do zapisanej współrzędnej `z_c`. To sprawia, że cały wydrukowany obiekt porusza się razem z blatem.

### 3.4 Stałe i Zmienne Sterujące – Rola i Znaczenie

Aplikacja wykorzystuje szereg zmiennych globalnych oraz stałych, które kontrolują zarówno logikę drukarki 3D, jak i aspekty wizualne oraz techniczne działania symulacji.

- **Stałe**: wartości zdefiniowane raz i niezmiennie w czasie działania programu. Zapisane wielkimi literami, np. `ROZMIAR_KOSTKI`, `NEUTRAL_Y_SUWAKA`,
- **Zmienne globalne**: aktualizowane dynamicznie w trakcie działania aplikacji – np. pozycja głowicy, status drukowania, animacje.

### 3.4.1 Główne stałe techniczne

- ROZMIAR\_KOSTKI = 0.02: rozmiar jednej "kostki" w jednostkach sceny OpenGL (szerokość, wysokość i głębokość pojedynczego bloku).
- NEUTRAL\_Y\_SUWAKA, NEUTRAL\_X\_GLOWICY, NEUTRAL\_Z\_BLATU: pozycje neutralne poszczególnych elementów – używane do resetowania drukarki do stanu początkowego.
- STALA\_PRZESUNIECIA\_DYSZY, STALA\_PRZESUNIECIA\_ATRAMENTU: precyzyjne przesunięcia pozycji końcówki i dyszy względem głowicy i suwaka. Dzięki nim blok pojawia się dokładnie tam, gdzie powinien.
- OPOZNIENIE\_KROKU, OPOZNIENIE\_USUWANIA: parametry czasowe kontrolujące szybkość drukowania i usuwania kostek.
- SZYBKOSC\_RECZNEGO\_DRUKOWANIA: maksymalna liczba bloków drukowanych na sekundę w trybie ręcznym.

### 3.4.2 Zmienne dynamiczne (globalne)

- x\_pos\_glowicy, y\_pos\_suwaka, z\_pos\_blatu: Aktualne współrzędne głównych części drukarki w przestrzeni 3D.
- pozycja\_neutralna, numer\_etapu: Flagi sterujące animacją powrotu do pozycji startowej.
- drukowanie, generator: Obsługa trybu automatycznego – drukowanie = True oznacza trwający proces, generator dostarcza kolejne komendy.
- cubes: Zbiór zawierający wszystkie wydrukowane klocki. Zaimplementowany jako set, aby uniknąć duplikatów.
- anim\_usuwania: Obsługa animacji usuwania wydruku – kasowanie po jednej kostce co 0.1 sekundy.
- tryb\_renderowania: Tryby renderowania – np. przełączenie na widok swobodny (bez drukarki) przy użyciu klawisza E.
- czas\_ostatniego\_bloku, czas\_ostatniego\_usunięcia: Znaczniki czasu używane do kontroli częstotliwości wykonywania operacji.

## 3.5 Wyświetlanie Pozycji Atramentu

Aby zapewnić użytkownikowi wgląd w aktualne położenie głowicy drukującej oraz precyzyjne śledzenie miejsca nanoszenia materiału, w projekcie zaimplementowano **dynamiczne wyświetlanie współrzędnych atramentu**.

Współrzędne są obliczane w czasie rzeczywistym i przedstawiają punkt w przestrzeni, w którym aktualnie znajduje się końcówka dyszy (czyli miejsce, gdzie pojawia się nowy blok). Dane te prezentowane są w formacie:

```
1 Atrament XYZ: (x, y, z)
```

### 3.5.1 Fragment kodu wyświetlającego pozycję atramentu

```
1      # WYŚWIETLANIE FPS
2      ink_pos_font = pygame.font.Font(None, 28)
3      ink_pos_text = f"Atrament XYZ: ({x_pos_glowica:.3f}, {y_pos_suwaka-
4      stala_przesuniecie_a_tramentu:.3f}, {stala_przesuniecie_wzgledem_srodka:.3f})
5
6      ink_surface = ink_pos_font.render(ink_pos_text, True, (0, 0, 0), (255,
7      255, 255, 180))
8      ink_text_data = pygame.image.tostring(ink_surface, "RGBA", True)
9      ink_text_x = 10
10     ink_text_y = text_y - ink_surface.get_height() - 5
11     glRasterPos2i(ink_text_x, ink_text_y)
12     glDrawPixels(ink_surface.get_width(), ink_surface.get_height(), GL_RGBA,
13     GL_UNSIGNED_BYTE, ink_text_data)
```

### 3.5.2 Wyjaśnienie działania kodu

- `ink_pos_font`: Obiekt czcionki utworzony przez Pygame (`Font(None, 28)`) oznacza domyślną czcionkę o rozmiarze 28).
- `ink_pos_text`: Tekst zawierający współrzędne X, Y, Z atramentu. Pozycje są zaokrąglane do 3 miejsc po przecinku.
- `ink_surface`: Renderowany tekst jako powierzchnia Pygame gotowy do konwersji do formatu graficznego.
- `ink_text_data`: Surowe dane RGBA utworzone z powierzchni Pygame potrzebne, by przesłać dane tekstowe do OpenGL.
- `ink_text_x, ink_text_y`: Współrzędne ekranu (w pikselach), gdzie tekst zostanie umieszczony w trybie 2D.
- `glRasterPos2i(x, y)`: Ustawia pozycję kursora rastrowego w OpenGL punkt startowy do rysowania tekstu.
- `glDrawPixels(...)`: Funkcja OpenGL, która bezpośrednio wstawia piksele (tekst) do bufora ekranu wykorzystywana do wyświetlania napisów i grafik 2D.

### 3.5.3 Obliczanie współrzędnych atramentu

- **X**: `x_pos_glowica` - aktualna pozycja głowicy w poziomie.
- **Y**: `y_pos_suwaka - stala_przesuniecie_a_trament` - faktyczne położenie końcówki dyszy względem suwaka (uwzględnia offset).
- **Z**: `stala_przesuniecie_wzgledem_srodka` - ponieważ głowica nie porusza się w osi Z, ta wartość jest stała (pozycja wydruku ustalana przez blat).

## 3.6 Wyświetlanie Licznika FPS

Wizualizacja liczby klatek na sekundę (FPS — Frames Per Second) pozwala użytkownikowi ocenić wydajność aplikacji 3D w czasie rzeczywistym. System został zaimplementowany w oparciu o bibliotekę `pygame` i może być realizowany na dwa sposoby — manualnie oraz przy użyciu wbudowanej funkcji `clock.get_fps()`.

### 3.6.1 Podejście manualne (starsze)

W pierwotnej wersji systemu licznik FPS był liczony ręcznie, poprzez analizę czasów renderowania kolejnych klatek w buforze czasowym:

```
1  # Zmienne do sledzenia i wyswietlania FPS
2  czas_klatki = []
3  odswiezanie_fps = 1.0 # sekundy
4  czas_ostatniego_odswiezenia = time.time()
5  displayed_fps = 0
6
7  while True:
8      aktualny_czas_fps = time.time()
9      czas_klatki.append(aktualny_czas_fps)
10
11     while czas_klatki and czas_klatki[0] < aktualny_czas_fps - (
12         odswiezanie_fps + 0.5):
13         czas_klatki.pop(0)
14
15     if aktualny_czas_fps - czas_ostatniego_odswiezenia > odswiezanie_fps:
16         if len(czas_klatki) > 1:
17             roznica_czasow = czas_klatki[-1] - czas_klatki[0]
18             if roznica_czasow > 0:
19                 displayed_fps = (len(czas_klatki) - 1) / roznica_czasow
20         else:
21             displayed_fps = 0
22         czas_ostatniego_odswiezenia = aktualny_czas_fps
```

### 3.6.2 Obecne podejście: clock.get\_fps()

W wersji aktualnej zastosowano prostsze i wbudowane rozwiązanie:

```
1  fps_text_surface = fps_font.render(f"FPS: {clock.get_fps():.2f}", True, (0,
2  0, 0), (255, 255, 255, 180))
```

Wartość `clock.get_fps()` pobierana jest bezpośrednio z obiektu zegara `pygame.time.Clock()`, który automatycznie oblicza średnią częstotliwość renderowania od ostatniego `tick()`.

## 4 Mechanizmy Animacji i Sterowania

### 4.1 Animacja Drukowania Sześcianu

Funkcja `generator_szescianu(n)` odpowiada za generowanie instrukcji niezbędnych do wydrukowania pełnego sześcianu o długości krawędzi  $n$  (w jednostkach logicznych drukarki, tj. liczbie małych bloków).

#### 4.1.1 Układ przestrzenny i orientacja osi

Wygenerowany sześcian ma strukturę opartą na 3-wymiarowej siatce o rozmiarze  $n \times n \times n$ , zorientowanej względem osi:

- **X (prawo–lewo)** – kolejne kolumny w warstwie,
- **Z (głębina)** – warstwy poziome w „plastrze”,
- **Y (wysokość)** – kolejne „plastry” od dołu do góry.

Sześcian budowany jest w kolejności:

**Y (plastry)  $\rightarrow$  Z (warstwy w plastrze)  $\rightarrow$  X (bloki w warstwie)**

#### 4.1.2 Komendy

Generator zwraca krotki z komendami, np.:

- ("USTAW\_GLOWICA\_X",  $x_{\text{pos}}$ ): Ustawia pozycję głowicy.
- ("USTAW\_BLAT\_Z",): Ustawia pozycję blatu.
- ("USTAW\_SUWAK\_Y",): Ustawia pozycję suwaka (podnosi lub obniża).
- ("PRINT\_BLOCK", ( $x, y, z$ )): Dodaje współrzędne nowego bloku do zbioru cubes.
- ("FINISHED",): Sygnalizuje koniec drukowania.

#### 4.1.3 Struktura generatora

Generator oparty jest na trzech zagnieżdżonych pętlach:

```
1 for k in range(n):      # os Y (plastry od dołu do góry)
2     for j in range(n):  # os Z (warstwy)
3         for i in range(n):      # os X (bloki w warstwie)
```

Każdy przebieg wewnętrznej pętli prowadzi do:

1. ustawienia pozycji głowicy (USTAW\_GLOWICA\_X),
2. zapisania pozycji bloku do wydruku (PRINT\_BLOCK).

#### 4.1.4 Matematyczna Analiza Funkcji

Funkcja `generator_szescianu(n)` generuje sześcian o wymiarach  $n \times n \times n$ , składający się z jednostkowych bloków o rozmiarze `ROZMIAR_KOSTKI` równym 0,02. Każdy blok reprezentowany jest przez środek sześcianu w przestrzeni trójwymiarowej jako współrzędna  $(x, y, z)$ .

- Oś X (prawo-lewo):

$$x_i = \left(i - \frac{n-1}{2}\right) \cdot \text{ROZMIAR\_KOSTKI}$$

gdzie  $i \in \{0, 1, \dots, n-1\}$  to indeks bloku w danym wierszu, a  $n$  to liczba bloków w jednej krawędzi. Wyrażenie  $(n-1)/2$  przesunęło układ współrzędnych tak, aby blok środkowy (lub środek między dwoma środkowymi w przypadku parzystego  $n$ ) leżał w punkcie  $x = 0$ , co zapewnia symetrię względem osi X.

Podobne obliczenia wykonywane są dla osi Y i Z:

- Oś Y (plastry od dołu do góry):

$$y_k = y_{\text{start}} + k \cdot \text{ROZMIAR\_KOSTKI}$$

gdzie  $k \in \{0, 1, \dots, n-1\}$  i  $y_{\text{start}} = -0.86 - \text{ROZMIAR\_KOSTKI}$ .

- Oś Z (warstwy w głębi sceny):

$$z_j = j \cdot \text{ROZMIAR\_KOSTKI}$$

gdzie  $j \in \{0, 1, \dots, n-1\}$ .

W ten sposób współrzędne  $(x, y, z)$  każdego bloku są dokładnie określone w przestrzeni 3D, a cały sześcian zbudowany jest z  $n^3$  bloków rozmieszczonych równomiernie w trzech osiach.

Dla każdej współrzędnej Y (czyli plastra), drukarka:

1. SET\_SUWAK\_Y: ustawia suwak na wysokość odpowiadającą  $y_k + \text{offset dyszy}$ ,
2. USTAW\_BLAT\_Z: drukuje wszystkie warstwy w osi Z, przesuwając blat (USTAW\_BLAT\_Z),
3. PRINT\_BLOCK: drukuje wszystkie bloki w danej warstwie w osi X,
4. RESET\_BLAT\_Z: po zakończeniu plastra – przesuwa blat do pozycji początkowej (RESET\_BLAT\_Z) o  $(n - 1) \cdot \text{ROZMIAR\_KOSTKI}$ .

W ten sposób powstaje pełny sześcian o równych krawędziach i dokładnym osadzeniu w przestrzeni.

## 4.2 Animacja Drukowania Kuli

Funkcja `generator_kuli` tworzy trójwymiarową sferę (kulę złożoną z małych sześcianów), o promieniu `r_promien` wyrażonym w jednostkach indeksów bloków, przy założeniu że:

- jeden blok ma rozmiar `ROZMIAR_KOSTKI = 0.02` jednostki OpenGL,
- kula jest wycentrowana wokół punktu  $(0, 0, 0)$ ,
- iteracja następuje po współrzędnych `k_x`, `k_y`, `k_z` w zakresie od `-r_promien` do `r_promien`. cubes.
- ("**FINISHED**",): Sygnalizuje koniec drukowania.

### 4.2.1 Warunek matematyczny przynależności punktu do kuli

Generator przechodzi po wszystkich punktach w sześciennym obszarze o rozmiarze:

$$(2 \cdot r\_promien + 1)^3$$

Dla każdego punktu o indeksach  $(k_x, k_y, k_z)$  sprawdzany jest warunek:

$$k_x^2 + k_y^2 + k_z^2 \leq r\_promien^2$$

Ten warunek pochodzi bezpośrednio z równania kuli w przestrzeni 3D:

$$x^2 + y^2 + z^2 \leq r^2$$

gdzie `k_x`, `k_y`, `k_z` są indeksami punktów w siatce sferycznej. Tylko te punkty, które spełniają ten warunek, zostają uznane za należące do kuli – i tylko dla nich drukarka generuje blok.

### 4.2.2 Przeliczanie indeksów na współrzędne fizyczne

Dla każdego zaakceptowanego punktu obliczane są rzeczywiste współrzędne  $(x, y, z)$  w scenie OpenGL:

- $X$  :

$$x = k_x \cdot \text{ROZMIAR\_KOSTKI}$$

- $Y$  :

$$y = y_{\text{start}} + (k_y + r\_promien) \cdot \text{ROZMIAR\_KOSTKI}$$

gdzie `y_start = -0.86` to punkt startowy najniższej warstwy kuli, a  $(k_y + r\_promien)$  przesuwa indeksy tak, by zaczynały się od 0.

- $Z$  :

$$z = k_z \cdot \text{ROZMIAR\_KOSTKI}$$

### 4.2.3 Kolejność budowania sfery

Sfera tworzona jest warstwowo:

1. **Po osi Y** – od dolnej warstwy do górnej ( $k\_y = r -$  do  $k\_y = r +$ ).
2. **W każdej warstwie Y** – kolejne plastry w osi Z ( $k\_z = r -$  do  $k\_z = r +$ ).
3. **W każdej warstwie Z** – skanowanie poziome w osi X ( $k\_x = r -$  do  $k\_x = r +$ ).

### 4.2.4 Liczba drukowanych bloków

Chociaż generator przeszukuje objętość sześcianu o rozmiarze:

$$(2 \cdot r\_promien + 1)^3$$

to realnie drukowanych bloków jest znacznie mniej, ponieważ tylko punkty wewnątrz kuli są brane pod uwagę.

Szacunkowa liczba drukowanych bloków jest proporcjonalna do objętości kuli:

$$\frac{4}{3}\pi r^3$$

## 4.3 Animacja Usuwania Wydruku

Aby umożliwić użytkownikowi wygodne zarządzanie modelem 3D oraz testowanie funkcji drukarki, zaimplementowano **animowaną sekwencję usuwania wcześniej wydrukowanych bloków**. Zamiast natychmiastowego wyczyszczenia całej sceny, kostki są usuwane **jedna po drugiej**, co nadaje procesowi wizualnej płynności i pozwala lepiej śledzić działanie systemu.

### 4.3.1 Mechanizm działania

Usuwanie inicjowane jest poprzez naciśnięcie klawisza DELETE:

```
1 if event.key == pygame.K_DELETE:
2     if cubes:
3         anim_usuwania = True
4         czas_ostatniego_usuwania = pygame.time.get_ticks()
5         print("Rozpoczeto animacje usuwania kostek.")
```

Zmienne kontrolujące animację:

- **anim\_usuwania**: flaga informująca, że trwa proces usuwania,
- **czas\_ostatniego\_usuwania**: znacznik czasu ostatniego kroku usuwania (w milisekundach),
- **OPOZNIENIE\_USUWANIA**: interwał pomiędzy usuwaniem kolejnych bloków, domyślnie 0.1 sekundy.

### 4.3.2 Struktura danych

Wszystkie wydrukowane bloki przechowywane są w zbiorze **cubes** jako unikalne współrzędne 3D. Dzięki strukturze typu **set**, usuwanie elementów odbywa się w czasie stałym:

```
1 removed = cubes.pop()
```

Po usunięciu bloku:

- zostaje on natychmiast usunięty z renderowania,
- na konsoli wyświetlana jest informacja: **Usunięto kostkę: (x, y, z)**,
- jeśli zbiór **cubes** jest pusty, animacja się kończy automatycznie.

### 4.3.3 Harmonogram animacji

Kod odpowiedzialny za animację znajduje się w głównej pętli renderującej:

```
1 if anim_usuwania:
2     teraz = pygame.time.get_ticks()
3     if teraz - czas_ostatniego_usuwania >= opoznienie_usuwania:
4         if cubes:
5             removed = cubes.pop()
6             print(f"Usunieto kostke: {removed}")
7             czas_ostatniego_usuwania = teraz
8         else:
9             anim_usuwania = False
10            print("Wszystkie kostki usuniete.")
```

Oznacza to, że:

- co 0.1 sekundy (lub więcej – zależnie od ustawień) usuwana jest jedna kostka,
- proces kończy się samoczynnie, bez potrzeby interwencji użytkownika.

### 4.4 Animacja Powrotu do Pozycji Neutralnej

Po zakończeniu procesu drukowania lub w wyniku ręcznego żądania (klawisz R), drukarka 3D automatycznie **wraca do pozycji neutralnej** – zdefiniowanej jako:

- `NEUTRAL_Y_SUWAKA = -0.6`
- `NEUTRAL_X_GLOWICY = 0.0`
- `NEUTRAL_Z_BLATU = 0.225`

Celem tej animacji jest przywrócenie systemu do stanu początkowego, co umożliwi np. rozpoczęcie nowego zadania lub dokładne wyzerowanie pozycji mechanicznych.

#### 4.4.1 Mechanizm działania

Proces powrotu odbywa się etapowo, poprzez funkcję `ustaw_pozycje_nautralna()`, która jest wywoływana cyklicznie w głównej pętli programu, o ile flaga `pozycja_nautralna` jest ustawiona na `True`.

```
1 if pozycja_nautralna:
2     ustaw_pozycje_nautralna()
```

#### 4.4.2 Logika etapów

Powrót do neutralnej pozycji podzielony jest na 3 etapy kontrolowane przez zmienną `numer_etapu`:

##### Etap 1: Podniesienie suwaka Y

```
1 if numer_etapu == 0:
2     if y_pos_suwaka < NEUTRAL_Y_SUWAKA:
3         y_pos_suwaka = min(y_pos_suwaka + 0.02, NEUTRAL_Y_SUWAKA)
4     else:
5         numer_etapu = 1
```

Suwak przesuwany się ku górze lub ku dołowi, aż osiągnie `-0.6`.

##### Etap 2: Ustawienie głowicy X na środek

```

1 elif numer_etapu == 1:
2     if x_pos_glowica > NEUTRAL_X_GLOWICY:
3         x_pos_glowica -= 0.01
4     elif x_pos_glowica < NEUTRAL_X_GLOWICY:
5         x_pos_glowica += 0.01
6     else:
7         numer_etapu = 2

```

Głowica precyzyjnie przemieszcza się w kierunku  $X = 0.0$ .

### Etap 3: Ustawienie wysokości blatu Z

```

1 elif numer_etapu == 2:
2     if z_pos_blatu > NEUTRAL_Z_BLATU:
3         z_pos_blatu -= 0.01
4     elif z_pos_blatu < NEUTRAL_Z_BLATU:
5         z_pos_blatu += 0.01
6     else:
7         pozycja_neutralna = False
8         numer_etapu = 0
9         # ewentualne rozpoczęcie nowego drukowania

```

Blat zostaje opuszczony lub podniesiony do wysokości  $Z = 0.225$ .

#### 4.4.3 Logika etapów

Po zakończeniu animacji, jeżeli wcześniej użytkownik wydał polecenie drukowania (`zadanie_do_druku_po_neutr` ustawione na "szescian" lub "kula"), system automatycznie rozpoczyna drukowanie

## 5 Wprowadzenie Elementów Uczenia Maszynowego: Nagrywanie i Odtwarzanie Akcji

W ramach projektu rozszerzono funkcjonalność drukarki 3D o proste mechanizmy "**uczenia maszynowego**", umożliwiające **nagrywanie manualnych interakcji użytkownika i ich późniejsze automatyczne odtwarzanie**. Nie jest to uczenie maszynowe w klasycznym sensie algorytmów opartych na danych i predykcjach, lecz implementacja systemu do **zapisu i odtwarzania sekwencji ruchów**, co stanowi podstawę dla bardziej zaawansowanych systemów automatyzacji.

### Mechanizm Działania:

#### 1. Tryb Nagrywania (klawisz /):

- Po aktywacji trybu nagrywania (`tryb_nagrywania = True`), aplikacja zaczyna **rejestrować wszystkie manualne ruchy użytkownika** (sterowanie głowicą, suwakiem, blatem) oraz **momenty dodawania bloków**.
- Każda zarejestrowana akcja jest przechowywana w liście `nagrane_ruchy` jako krotka zawierająca **typ akcji** (np. `'MOVE_X_GLOWICA'`, `'PRINT_BLOCK'`), **wartość** (np. nowa pozycja, współrzędne bloku) oraz **czas, który upłynął od rozpoczęcia nagrywania**. Pozwala to na **precyzyjne odtworzenie akcji z zachowaniem oryginalnego tempa**.
- Dezaktywacja trybu nagrywania (/) **zatrzymuje zapis i informuje o liczbie zarejestrowanych akcji**.
- Zarejestrowane ruchy są **buforowane w pamięci**. Dla bardzo długich sekwencji ruchów, **warto rozważyć zapisywanie ich do pliku**.

- Precyzja odtwarzania **zależy od częstotliwości próbkowania ruchów podczas nagrywania** (czyli jak często aktualizowana jest lista `nagrane_ruchy`).

## 2. Tryb Odtwarzania (klawisz p):

- Przed rozpoczęciem odtwarzania, drukarka automatycznie **wraca do pozycji neutralnej**. Ma to na celu **zapewnienie spójności i powtarzalności odtwarzanych sekwencji**, niezależnie od stanu początkowego drukarki.
- Po osiągnięciu pozycji neutralnej, system zaczyna **iterować przez listę nagrane\_ruchy**.
- **Odtwarzanie odbywa się z zachowaniem zarejestrowanych odstępów czasowych**. Oznacza to, że każda akcja jest wykonywana dopiero w momencie, gdy od początku odtwarzania upłynął czas równy zarejestrowanemu `czas_akcji`.
- W przypadku akcji `'PRINT_BLOCK'`, system **dodaje kostkę** na zarejestrowanej pozycji.
- **Monitorowanie postępu odtwarzania** jest wyświetlane na ekranie, informując o aktualnie odtwarzanej akcji (`TRYB ODTWARZANIA [aktualny_krok/wszystkie_kroki]`).

## 6 Oświetlenie, Shadery i Cienie

Projekt wykorzystuje podstawowy model oświetlenia dostępny w tzw. fixed-function pipeline OpenGL, oparty na klasycznym modelu Phong'a. Pozwala on na uzyskanie trójwymiarowego wyglądu obiektów poprzez zastosowanie trzech składników światła: ambient, diffuse oraz specular.

- **Ambient:** Odpowiada za ogólne oświetlenie sceny, niezależne od pozycji światła. W tym projekcie zastosowano wartość `[0.2, 0.2, 0.2, 0.2]` dla delikatnego tła.
- **Diffuse:** Komponent ten uwzględnia orientację powierzchni względem źródła światła, co tworzy wrażenie plastyczności i wypukłości. Dla światła rozproszonego użyto wartości `[0.8, 0.8, 0.8, 1.0]`.
- **Specular:** Symuluje błyszczenie na gładkich powierzchniach, dodając realizmu, szczególnie przy jasnym świetle i dużym połysku. Dla odbić lustrzanych zastosowano `[1.0, 1.0, 1.0, 1.0]`.

W programie skonfigurowano źródło światła z wykorzystaniem funkcji `glLightfv`, przypisując wartości `pozycji`, `ambientu`, `diffuse`, a opcjonalnie również `specular`. Pozycja źródła światła została ustalona na `[0.0, 4.0, 0.0, 1.0]`. Powierzchnie obiektów mają przypisane materiały za pomocą `glMaterialfv`, co umożliwi interakcję z oświetleniem – np. odbłaski lub brak światła na nieoświetlonych ścianach. Każda ściana sześcianu ma także przypisany wektor normalny, co jest kluczowe dla prawidłowego działania oświetlenia kierunkowego.

Cienie są generowane za pomocą techniki **shadow mapping**:

1. Renderowanie sceny z perspektywy światła do bufora głębi.
2. Użycie macierzy projekcji świetlnej w shaderze fragmentów do sprawdzenia, czy piksel jest w cieniu.

Efekt cieni zwiększa głębię wizualną, szczególnie na blacie i wydrukowanych obiektach.

Obecna konfiguracja oświetlenia znacząco poprawia czytelność i estetykę sceny 3D, czyniąc wizualizację bardziej intuicyjną i realistyczną.